

智能体优先的营销平台系统方案

基于现有模块组合：

- 采集：MediaCrawler
- 分析：video-analyzer
- 分发：Mixpost（仓库内 `./mixpost`）
- 生成：商用模型 / Ollama（LLM + 文生视频可选）

文档定位与阅读顺序：先读 技术方案.md（总体与原则）→ 再读 开发方案.md（接口/数据/状态机的唯一口径）→ 最后执行 实施路径.md（周计划与验收清单）。如有冲突，以 开发方案.md 为准。

0. 设计目标与约束

目标

- **稳定可靠**：采集/分析/生成/分发都是长链路任务，要求可重试、可恢复、可追溯
- **智能体模式**：支持多角色 agent 协作、可插拔工具、可配置 workflow、人审闭环
- **可扩展**：平台与模型解耦；新增平台、新增分析、新增视频生成方式不改核心
- **合规优先（医美强需求）**：内容合规检测 + 风险提示 + 审批流关键约束

现状

- 你现有组件多为独立工程（爬虫/分析/分发），必须用标准化接口统一起来
- 任务链路长：必须要有队列 + 状态机 + 事件驱动，否则必炸

1. 总体架构：Agent Orchestrator + Tool Services + Event Bus

1.1 核心思路 (Orchestrator 边界收紧)

把整个系统拆成三层，严格遵守三条铁律：

1. Agent 编排层 (Orchestrator)

- **只能做**：调度、状态流转、路由分发
- **不能做**：任何业务判断（如评分、策略选择、文案生成），只负责“下一步去哪”

2. Smart Agents (脑)

- **只能做**：决策、策略生成、复杂判断（所有 if/else 业务逻辑都在这）
- **不能做**：底层执行细节（如 HTTP 请求、文件读写）

3. Tool Services (手/执行器)

- **只能做**：执行具体任务（MediaCrawler, Mixpost），返回结果
- **不能做**：流程决策

1.2 推荐技术组合（默认可落地）

- **编排与状态机**：LangGraph（默认）+ Celery（异步执行）；Temporal 作为后续可选升级
- **任务队列/消息**：默认 Redis（Celery broker/result backend）；规模上来再评估 RabbitMQ/Kafka
- **存储**：
 - Postgres：业务数据（任务、项目、客户、审批）
 - Object Storage：视频/音频/帧图/封面（MinIO/S3）
 - Elasticsearch：检索（爆款库、脚本库、标签）
- **可观测（必须完整实现）**：
 - **分布式追踪**：OpenTelemetry SDK 集成到所有服务，统一使用 W3C TraceContext（traceparent / tracestate）传播
 - **请求关联**：对外 API 要求 x-request-id（缺省由入口网关生成并回显），服务间透传
 - **指标收集**：Prometheus + Grafana
 - **日志聚合**：Loki 或 ELK（可选）
- **工具协议**：优先做成 HTTP/gRPC，再往 MCP/插件化演进

先用 Celery/RQ + Postgres + MinIO + ES 快速跑起来；后面规模上来再升级队列/分布式。

2. “智能体系统”设计：角色、职责、协作方式

2.1 Agent 角色划分（建议 6 个核心 agent + 2 个守门 agent）

核心生产链路 agent

1. Scout Agent（情报采集）

- 调用 MediaCrawler：按关键词/账号/话题抓取
- 输出：候选内容清单 + 元数据

2. Judge Agent（爆款判定）

- 规则 + 模型：根据点赞/评论/完播（如可得）筛选“爆款候选”
- 输出：爆款候选队列 + 标签（项目/人群/风格）

3. Analyst Agent（结构拆解）

- 调用 video-analyzer：抽帧、镜头切分、字幕/音频要点
- LLM 总结：Hook、冲突点、背书、CTA、节奏、情绪曲线
- 输出：结构化“爆款公式卡片”（你平台最值钱的资产）

4. Writer Agent（脚本与文案）

- 输入：产品特性 + 爆款公式卡片 + 目标人群 + 平台偏好
- 输出：标题/口播/分镜/镜头指令/评论区话术/私信话术

5. Producer Agent（视频生产）

- 方式 A：模板化剪辑（拼素材、加字幕、封面、BGM）
- 方式 B：调用文生视频/数字人
- 输出：成片、多比例版本、封面、字幕文件、发布参数

6. Publisher Agent（分发运营）

- 定位：**策略与调度者
- 工具：**调用 Mixpost (Publisher Tool) 执行发送
- 职责：**决定发哪个号、什么时间发、失败怎么重试。Mixpost 只是执行器，不存核心数据。

守门与闭环 agent（医美刚需）

7. Compliance Agent（合规审查 - 双层架构）

- Layer 1: Rule-based Core (硬规则)**
 - 禁用词表、法规映射、句式模板（不论模型怎么说，这个不通过就是不通过）
- Layer 2: LLM Assistant (软助手)**
 - 负责风险解释、文案柔性改写、人审辅助建议
- 原则：**规则在前，模型在后。

8. Learning Agent（学习与策略迭代）

- Phase 1-2 (前 6 个月)：**
 - ✗ 禁用 RL / 自动改 Prompt**（避免过拟合与噪声放大）
 - ✓ 仅做：**权重统计（哪个 Pattern 成功率高）、平台维度对比报表
- 职责：**生成新版本 PatternCard（不覆盖旧版），供人工确认。

2.2 协作方式：Graph + 事件驱动 + 人审节点

- Graph（状态机）**保证“可恢复、可追溯”
- 事件驱动**保证“模块解耦、可扩展”
- 人审节点**保证“医美合规与最终质量”

2.2.1 LangGraph 节点分类规范 (P0)

为避免“策略 if/else”混入 Graph 路由逻辑，所有节点必须明确分类：

节点类型	职责	允许操作	审计要求
Router Node	仅路由/调度/状态流转	读 Step 状态、写 Step 状态、调用 Tool	无特殊要求
Decision Node	业务决策 (LLM 调用)	调用 LLM、生成内容	必须落审计： model, prompt_hash, input_refs, output_hash

约束: Decision Node 的 LLM 调用必须经过 `llm-service`，且每次调用必须生成可回放的审计历史。

3. 标准化“工具接口”封装（关键：把现有模块产品化）

你现有 3 个开源组件要变成生产系统的一部分，核心是：统一工具协议与输出结构。

3.1 Tool API 规范（建议）

每个工具服务都必须遵守统一契约 (P0):

- `POST /v1/jobs`：创建任务（通过 `job_type` 区分：`crawl` | `analyze` | `llm_generate` | `video_produce` | `publish` | `metrics`，并要求 `idempotency_key`）
- `GET /v1/jobs/{job_id}`：查询状态
- `GET /v1/jobs/{job_id}/result`：取结果（含 `artifacts[]`）

标准返回（最小字段）：

- `job_id`
- `status`（`PENDING/RUNNING/SUCCEEDED/FAILED`）
- `error`（失败时：`code/message/retryable`）
- `artifacts[]`（产物：视频、字幕、json 特征等，统一使用 `asset://` 逻辑 URI）

3.2 你现有模块如何封装

A) MediaCrawler Tool Service

- 源码位置：`./MediaCrawler` (Python)
- 能力：`crawl_by_keyword(platform, keyword, time_range, limit)`
- `crawl_by_user(platform, user_id, limit)`
- 输出（统一字段）：
 - `content_id, platform, author_id, url, title, metrics, publish_time, raw_payload_ref`

B) video-analyzer Tool Service

- 源码位置：`./video-analyzer`
- 能力：`analyze_video(video_asset_ref, options)`：抽帧、镜头、ASR（如有）、节奏等（运行时通过 `assets-service` 解析到物理 URL）
- 输出：
 - `shots[], keyframes[], transcript, entities, audio_features, visual_features`

C) Mixpost Tool Service (Publisher Tool)

- 源码位置： ./mixpost (Laravel)
- **定位**：纯粹的执行管道，降级为 Publisher Tool。
- 集成方式（推荐落地顺序）：
 - **MVP**： publisher-service 先按 **Stub Publish** 落地（写 `publish_records.status=PENDING_MANUAL`），人工在 Mixpost UI 发布后通过 backfill 接口回填 `post_id/published_at`
 - **Phase 3**： Fork Mixpost 增加最小 REST API（Token 鉴权）或封装 Artisan Command 调用，接入自动发布
- 能力： `publish(post_payload, accounts[], schedule_time)`
- `publish_status(platform_post_id)`
- **注意**：不依赖 Mixpost 存发布的长期数据，核心 `PublishRecord` 存在我们自己的 Postgres。

D) LLM / Ollama Tool Service

- 能力： `generate_copy(product_brief, pattern_card, constraints)`
- `rewrite_for_compliance(copy, rules)`
- `summarize_pattern(analyzer_output)`
- 输出：
 - `copy_variants[], shotlist, risk_notes`

E) Video Generation Tool Service（可多实现）

- 实现 1：模板化剪辑（强烈建议先做这个，最稳）
- 实现 2：数字人口播
- 实现 3：文生视频（可选）
- 所有实现都统一输出：
 - `video_uri[]`（不同平台比例/时长）
 - `cover_uri`
 - `subtitle_srt_uri`
 - `edit_recipe_json`（非常重要：可回放、可复用）

4. 两条核心工作流（MVP + 扩展）

4.1 工作流 1：爆款库构建（每天自动跑）

目标：形成“爆款资产库 + 方法论卡片库”

1. **Scout Agent** → 采集候选内容
2. **Judge Agent** → 基于指标筛选 + 分类
3. **Analyst Agent** → 调 video-analyzer + LLM 抽象爆款结构
4. **Compliance Agent** → 标注风险（用于后续生成约束）

5. 入库：Pattern Card / Hook 库 / CTA 库 / 镜头库

产物（你的壁垒）：

- PatternCard （结构化爆款模板）
- ProjectTags （项目与人群映射）
- PlatformStyle （平台偏好特征）

4.2 工作流 2：给定产品 → 生成 → 审核 → 发 → 回流

目标：客户拿到可用内容，并形成数据闭环

1. **Writer Agent**：基于产品 brief + PatternCard 生成多版本脚本
2. **Compliance Agent**：
 - 低风险：自动通过
 - 中高风险：进入人审（运营/法务/医生）
3. **Producer Agent**：生成视频（模板剪辑/数字人/文生视频）
4. **Publisher Agent**：调用 Mixpost 发布
5. **Learning Agent**：抓取数据回流，更新策略

5. 数据模型：把“内容资产化”（扩展性关键）

建议你从一开始就把数据做成“可资产化”的结构，否则后面很难形成壁垒。

5.1 核心实体（表/索引）

- ContentItem：采集到的视频/帖子
- PatternCard：爆款结构卡
 - version：版本号 (v1, v2...)
 - valid_from：生效时间
 - valid_to：失效时间
 - deprecated_reason：废弃原因 (监管变更/平台算法变更)
- ProductBrief：产品/项目简述（机构输入）
- Script：文案脚本（版本、多平台适配）
- VideoAsset：成片、封面、字幕、配方 (recipe)
- PublishRecord：发布记录（平台ID、账号、时间）
- PerformanceMetric：表现数据（曝光/互动/私信/线索）
- ComplianceReport：合规报告（风险等级、触发规则、改写）

5.2 “Recipe（配方）”机制（强烈建议）

每个视频生成都存一份 edit_recipe_json：

- 用了哪些素材（URI）
- 字幕样式、封面模板、BGM
- 镜头顺序、裁剪参数、时长
- 平台适配参数

好处：可回放、可复刻、可批量变体生成，后续做 A/B 测试会很顺。

6. 可靠性设计（生产级必须有）

6.1 任务可靠性：幂等 + 重试 + 断点恢复 + 心跳

- 每个 tool 调用都必须带 idempotency_key （例如 task_id + step ）
- 失败可重试（区分可重试/不可重试）
- 状态持久化：每步产物落盘（视频、json、日志）

⚠️ Outbox 语义钉死 (P0 - MVP 最小闭环):

- **投递模式:** 事务内写表 + Dispatcher 定时扫描投递（至少一次）
- **消费幂等:** 消费端以 event.id 去重（Redis SETNX + TTL=10min 或 DB unique）
- **失败补偿:** Dispatcher 投递失败 → 指数退避 ($\text{power}(2, \text{retry_count})$) + Dead Letter (status='DEAD')
- **详细规范:** 参见 开发方案.md Part 4

⚠️ 心跳与 Lease 续租机制 (P0)

为解决 Worker 崩溃导致的任务卡死，必须引入 Lease (租约) 机制：

1. **抢占 Step:** Worker 领取任务时，原子写入 lease_owner + lease_expires_at (NOW + 30s)。
2. **心跳续租:** Worker 每 10s 发送心跳，更新 lease_expires_at = NOW + 30s。
 - **约束:** 只有 lease_owner 匹配且未过期的 Worker 才能续租。
3. **死信监控:** Orchestrator 每 60s 扫描 lease_expires_at < NOW 的 RUNNING 任务，标记为 HEARTBEAT_TIMEOUT 。

⚠️ 超时判定优先级 (硬规范 P0):

场景	条件	判定结果	处理动作
业务超时 (SLA)	deadline_at < NOW	Job Failed	优先中止任务，人工介入或重试
Worker 失联 (活性)	lease_expires_at < NOW	Worker Lost	触发 HEARTBEAT_TIMEOUT 重试，允许被其他 Worker 抢占

规则：当两者同时满足时， deadline 优先级更高（即先判定业务失败）。

错误分类与重试策略:

- **retry_policy:**
 - **EXP_BACKOFF** : 指数退避 (1s, 2s, 4s...) -> 适用于 **TOOL_429** , **NETWORK_ERROR**
 - **FIXED_INTERVAL** : 固定间隔 -> 适用于 **HEARTBEAT_TIMEOUT**
 - **NONE** : 业务校验失败 -> 适用于 **NON_RETRYABLE_VALIDATION**

6.2 乐观锁与并发控制（必须实现）

防止多个 Worker 并发更新同一 Step 导致数据覆盖:

⚠ **DDL 唯一真理源:** 参见 开发方案.md 3.1 任务调度

`job_steps.version INT DEFAULT 0` — 以开发方案 DDL 为准

-- 所有更新必须带版本校验（示例）

```
UPDATE job_steps
SET status = 'SUCCEEDED', output_data = $data, version = version + 1
WHERE id = $step_id AND version = $expected_version;
```

-- 若受影响行数为 0, 说明被其他进程修改, 必须重新读取后重试

6.3 人审（Human-in-the-loop）

- **医美一定要做:**
 - 文案审核: 禁用词/夸大/承诺效果/对比广告
 - 视觉审核: 前后对比图、案例真实性提示
- **流程上:** 审批通过才能进入 Producer/Publisher
- ⚠ **超时处理（可执行契约）:**
 - 当 Step 从 **RUNNING** -> **WAITING_FOR_APPROVAL** 时, 必须原子写入:
 - `deadline_at = NOW() + INTERVAL '72 hours'` （审批最终截止）
 - `entered_waiting_at = NOW()` （首次进入审批态的时间）
 - `last_reminded_at = NULL` （重置提醒状态）
 - 审批提醒扫描（每 10 分钟）:
 - 条件: `status = 'WAITING_FOR_APPROVAL'`
 - 且 `NOW() - COALESCE(last_reminded_at, entered_waiting_at) >= INTERVAL '24 hours'`
 - 动作: 发送通知 + `UPDATE ... SET last_reminded_at = NOW()`
 - 审批终止扫描:
 - 条件: `status = 'WAITING_FOR_APPROVAL' AND deadline_at < NOW()`
 - 动作: `UPDATE ... SET status = 'ABORTED', error_code = 'APPROVAL_TIMEOUT'` （必须审计记录）
 - **注:** 此机制依赖 `job_steps` 表的 `entered_waiting_at` 和 `last_reminded_at` 字段

6.3.1 ApprovalScanner 实现规范 (P0)

Orchestrator 必须启动 ApprovalScanner 定时任务

参数	值	说明
扫描间隔	5 分钟	Cron: */5 * * * *
提醒基准	entered_waiting_at	唯一时间基准，避免多次提醒
提醒间隔	24h	距离上次提醒或进入时间
终止截止	72h	距离 entered_waiting_at

```
-- 提醒扫描 (Reminder)
UPDATE job_steps SET last_reminded_at = NOW()
WHERE status = 'WAITING_FOR_APPROVAL'
      AND NOW() - COALESCE(last_reminded_at, entered_waiting_at) >= INTERVAL '24 hours'
      AND deadline_at > NOW()
RETURNING id, job_id; -- 对返回结果发通知

-- 终止扫描 (Abort)
UPDATE job_steps
SET status = 'ABORTED', error_code = 'APPROVAL_TIMEOUT',
    finalized_at = NOW(), version = version + 1
WHERE status = 'WAITING_FOR_APPROVAL' AND deadline_at < NOW();
```

6.4 审计与追溯

- 你要做到：“这条视频为什么这么写？”(引用了哪个 PatternCard，哪些素材)
- “限流/违规后如何回溯？”(对应 ComplianceReport 与发布版本)

7. 扩展机制：新增平台/新增模型/新增 agent 不改核心

7.1 插件化的 Tool Registry

- 所有工具注册到 Tool Registry（类似插件市场）
- Agent 只依赖工具“能力名称”，不依赖实现
- 例如： video.generate 可以对应 3 种实现（模板剪辑/数字人/文生视频）

7.2 多模型路由（商用模型 + Ollama）

- 做一个 Model Router：
 - 文案生成：商用 LLM（质量优先） / Ollama（成本优先） 可选
 - 合规改写：优先可控（规则+小模型），不建议完全靠大模型
 - 分镜与镜头：可用更强模型

- 所有请求打标签: cost, quality, risk

7.3 媒体资产 URI 抽象层（必须实现）

为避免 OSS 厂商迁移时全量更新数据，采用**逻辑 URI / 物理 URI 分离**：

```
{
  "asset_ref": "asset://{tenant}/video/abc123",           // 逻辑引用（存储在数据库）
  "resolved_url": "https://cdn.example.com/v/abc123.mp4" // 运行时解析
}
```

URI Resolver 服务 (位于 assets-service)：

⚠ **路由设计（避免 path 歧义 - P0）：**

- **唯一允许**: POST /v1/assets/resolve
- **Body**: {"asset_ref": "asset://{tenant}/{type}/{id}"}
- **禁止**: 任何形式的 GET 请求（包括 query param），以确保 Ref 字符完整性和避免日志泄漏。

支持配置不同 Backend: MinIO / AWS S3 / 阿里云 OSS / CDN

8. API 安全性设计（必须实现）

8.1 速率限制 (Rate Limiting)

所有对外 API 必须实现速率限制，返回标准 Header：

```
X-RateLimit-Limit: 100           # 每分钟最大请求数
X-RateLimit-Remaining: 45        # 剩余请求数
X-RateLimit-Reset: 1704614400    # 重置时间戳 (epoch seconds, UTC)
```

⚠ **429 响应必须返回 (P0)：**

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30                 # 建议等待秒数
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1704614400
```

多维度限流策略 (P1)：

- **Tenant 级 (Base)**: 防止租户滥用 (e.g., 1000 req/min)
- **User/IP 级 (Defense)**: 防止单用户被盗刷或恶意攻击 (e.g., 100 req/min/IP)

8.2 事件安全 (Webhook Security)

8.2.1 签名头与必填字段 (P0)

- 事件投递必须携带 Header: X-Agency-Signature: sha256=<hex>
- Body 必须包含 CloudEvents 标准字段:
 - id (全局唯一)
 - time (ISO8601 UTC, **统一使用 CloudEvents 标准字段名**)
 - type
 - data
- Body 中禁止包含 signature 字段 (签名只在 Header 中)

8.2.2 Canonicalization 规则 (P0, 跨语言必须一致)

用于签名的 canonical_string 定义为:

```
canonical_string = `${time}.${id}.${sha256(canonical_json_body)}`
```

其中:

- time: body.time 原样字符串 (例如 2026-01-08T10:00:00Z)
- id: body.id
- canonical_json_body: 对"整个 Body"做 JSON 规范化:
 - sort_keys=true
 - separators=(',', ':') (无空格)
 - ensure_ascii=false
 - Unicode 字符串 NFC 归一化 (如适用)

8.2.3 签名算法 (HMAC-SHA256)

```
X-Agency-Signature = "sha256=" + HMAC_SHA256(secret, canonical_string).hexdigest()
```

8.2.4 防重放 (P0)

消费者必须执行:

- 时间窗口: $\text{abs}(\text{now} - \text{body.time}) \leq 300\text{s}$, 否则拒绝
- 幂等去重: 使用 event.id 作为 key (Redis SETNX + TTL=10min), 重复则忽略 (返回 2xx)

8.3 敏感数据加密

8.4 认证与上下文 (Authentication & Context - P0)

Header 信任边界：

- 所有内部服务（Tool Services）**必须**信任并透传 `x-user-id` 和 `x-tenant-id`。
- **API Gateway (Orchestrator)** 负责从 OAuth2/JWT Token 解析这些字段并写入 Header。
- **禁止**前端/客户端直接发送这些 Header（Gateway 必须覆盖/丢弃客户端传来的伪造值）。

敏感操作权限：

- 对于 **人审 (Approve/Reject)**、**Backfill**、**规则更新** 接口，必须在 Gateway 层校验 RBAC 角色（如 `ROLE_ADMIN`，`ROLE_OPERATOR`）。
- **账号凭证 (OAuth Token)**：使用 AES-256-GCM 加密存储
 - **Key Management**: 必须支持 Key 轮换 (使用 `key_id` 标识版本)。
 - **Env**: 禁止硬编码，推荐使用 Vault 或 K8s Secrets。
- **API Keys**：仅存储 Hash，不可逆
- **日志脱敏**：禁止在日志中打印完整 Token、手机号、身份证

9. 一份“可执行”的落地里程碑（不拖泥带水）

Phase 1: 把三件事连起来（最小闭环）

- MediaCrawler → 入库（视频与元数据）
- video-analyzer → 产出结构化分析结果
- LLM → 生成脚本（先不做视频生成也行）

交付：

- ☒ 爆款库 + 脚本生成可用

Phase 2: 加上“合规 + 视频工厂”

- 合规规则库 (Rule-based) + LLM 助手 + 人审台
- 模板剪辑视频生成（字幕/封面/比例适配）
- recipe 落库

交付：

- ☒ 可交付给客户的“脚本+成片”闭环

Phase 3: 接入 Mixpost 工具 + 数据回流

- Mixpost 作为 Publisher Tool 发帖
- 回流指标采集（至少曝光/互动）
- **Learning Agent 上线 (Phase 1)**：只做权重统计与报表，不做自动调节，作为决策辅助。

交付:

-  “发布→数据→优化”的飞轮雏形

10. 前端架构 (基于 Yuxi-Know 最佳实践)

鉴于 Yuxi-Know 优秀的前端架构，本平台前端将复用其核心技术栈与设计模式。

10.1 技术栈选型

- **核心框架:** Vue 3.5+ (Composition API) + Vite 7+
-  **语言 (强制要求):** TypeScript (不再使用 JavaScript, 确保类型安全)
- **UI 组件库:** Ant Design Vue 4.x + Lucide Icons
- **状态管理:** Pinia (配合 pinia-plugin-persistedstate 做持久化)
- **数据可视化:**
 - ECharts: 基础报表与仪表盘
 - **Vue Flow (推荐):** 智能体编排画布 (API 更贴合 Vue 生态)
 - AntV G6: 复杂知识图谱可视化 (仅高级场景使用)
- **编辑器:**
 - md-editor-v3: 脚本/文案编辑
 - Markmap: 结构化思维导图总结

10.2 核心模块规划

1. Agent Workbench (智能体工作台)

- 采用 Chat-based UI (类 cursor) 与右侧 **Artifact View** (预览区) 并行布局。
- 左侧: 对话指令、任务提交。
- 右侧: 脚本预览、视频播放器、数据卡片。

2. Workflow Studio (编排画布)

- 使用 **Vue Flow** 实现拖拽式 Agent/Tool 编排 (推荐)。
- 备选: AntV G6 (仅当需要复杂子图/聚合时使用)。

3. Media Asset Library (素材库)

- 瀑布流布局 (Masonry) 展示视频/图片。
- 悬停快速预览 (Gif/Video)。